

Pipelined Processor

- ❖ Pipelining improves the throughput of a digital system.
- ❖ It is designed by subdividing the single-cycle processor into five pipeline stages.
- ❖ Five instructions can execute simultaneously, one in each stage.
- ❖ As each stage has only one-fifth of the entire logic, the clock frequency is almost five times faster.
- ❖ Hence, the latency of each instruction is ideally unchanged, but the throughput is ideally five-times better.

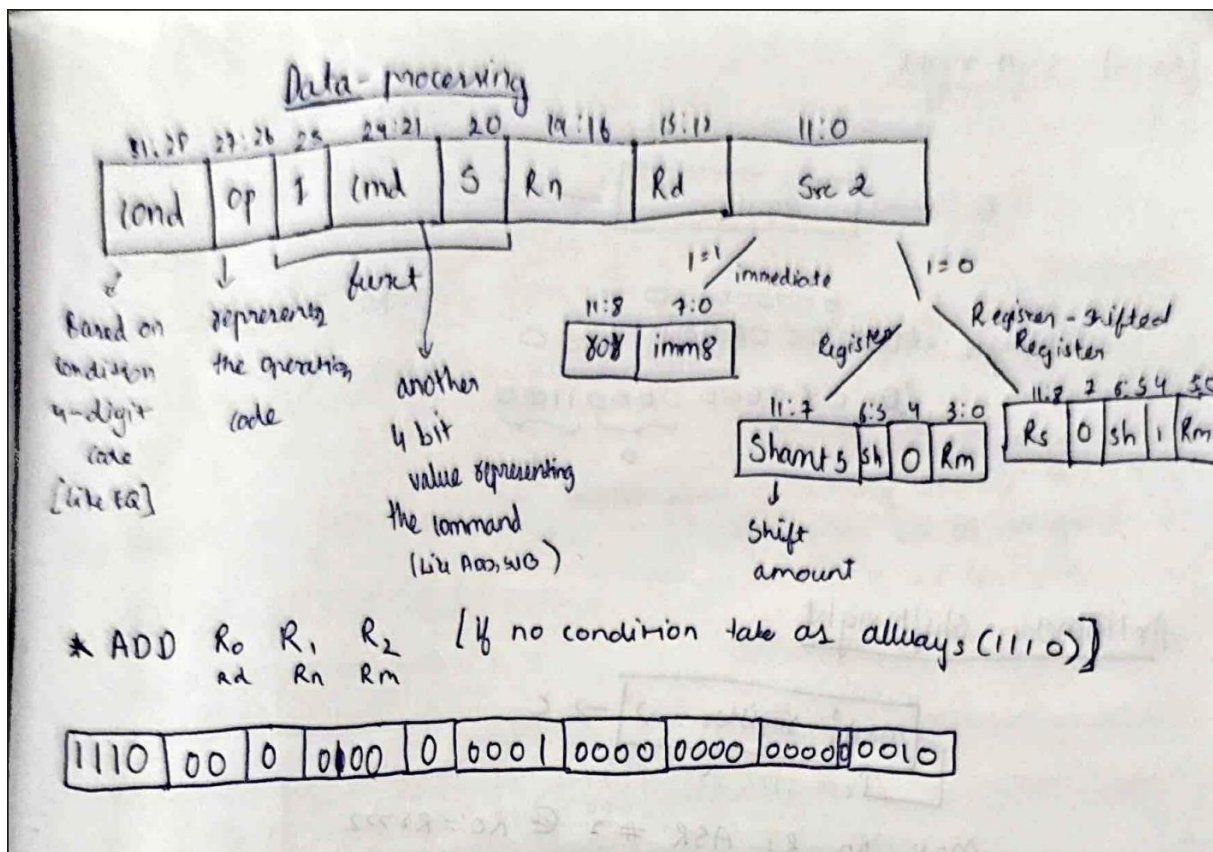
Pipelined Processor

Pipeline Stages

- 1) Fetch
 - ❖ Processor reads the instruction from instruction memory.
- 2) Decode
 - ❖ Processor reads the source operands from the register file and decodes the instruction to produce the control signals.
- 3) Execute
 - ❖ Processor performs a computation with the ALU.
- 4) Memory
 - ❖ Processor reads or writes the data memory.
- 5) Writeback
 - ❖ Processor writes the result to the register file, when applicable.

Pipelined Processor

- ❖ Each pipeline stage is represented with its five major component
 - 1) Instruction memory (IM),
 - 2) Register file (RF) read,
 - 3) ALU execution,
 - 4) Data memory (DM),
 - 5) Register file writeback



ADD Rd, Rn, Rm

ADD Rd, Rn #some immi value)

1) SUBEQS R₀, R₁, #2 [SUB → 0010, EQ → 0000]

0000	00	1	0010	1	0001	0000	0000	0000	0010
------	----	---	------	---	------	------	------	------	------

[if immediate value is more than 256 then we use (ROT) rotate right function]

2) ADD R₀ R₁ R₂ → This instruction is wrong because there is no status bit as we get a carry and zero value.

R₁ = 0x FFFFFFFF
 R₂ = 0x 00000001

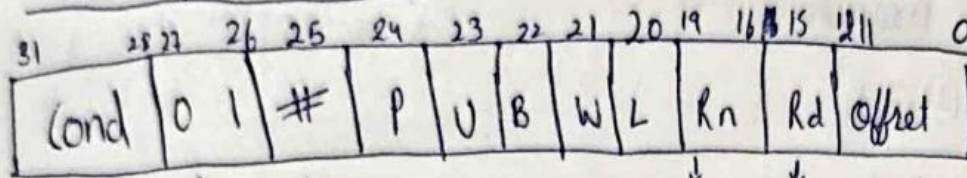
R ₁ =	1111	1111	1111	1111	1111	1111	1111	1111	1111
R ₂ =	0000	0000	0000	0000	0000	0000	0000	0000	0001
	0000	0000	0000	0000	0000	0000	0000	0000	0000

C = 1 Z = 1 [Carry flag is associated with Unsigned addition whereas, overflow flag is associated with signed addition]

3) ADD R₀, R₁, R₂, LSL #2

1110	00	0	0100	0	0001	0000	0001	00	0010
------	----	---	------	---	------	------	------	----	------

→ Data transfer Instructions



↓
Op
code

↓
offset
value

Base reg-
-stem destination in load, source in store

= 0

= 1

[immediate
value] → R₂[R₀ #4]

[it is 1 when it
is a register] → R₂[R₀ R₃ LSL #4]

11 0
12-bit immediate

11	7 6	5	4	3	0
#Shift		sh	0	Rm	

↓
Shift
amount

↓
type
shift

↓
offset
register

Flag bits

- L → Single bit, represent if it is Load or store

L = 0 → Store instruction

L = 1 → Load instruction

- W → W = 0 → no write back [!]
W = 1 → write back

- B → unsigned byte or word
B = 0 → word
B = 1 → byte

- U → increment or decrement (up or down)

U = 0 → subtract offset from base

U = 1 → add offset to the base

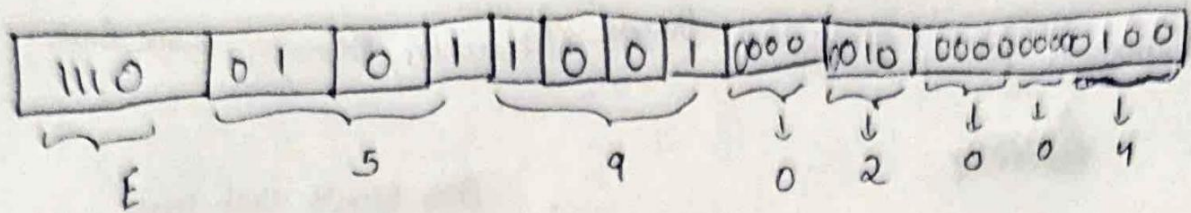
- P → postfix or prefix [postindex or pre-index]

P = 0 → Post

P = 1 → Pre

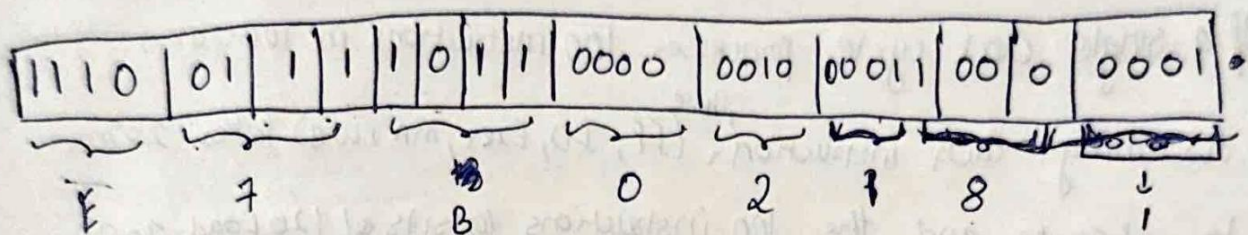
00	-LSL
01	-ASR
10	-ASR
11	-RR

Q) LDR R₂[R₀ #4]

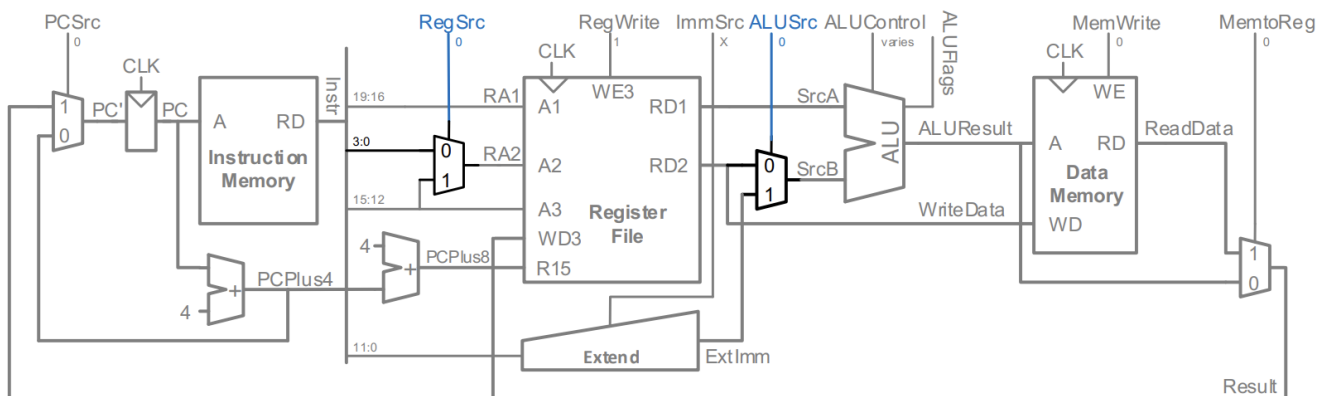


⇒ 0x E5902004 → hexadecimal format

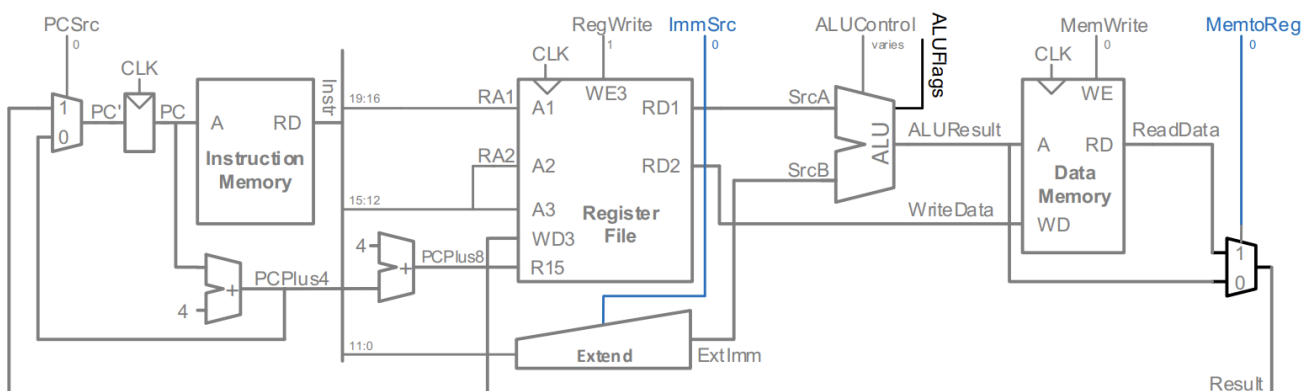
Q) LDR R₂[R₀, R₁, LSL #3]!



⇒ 0x E7B02181



ADD Rd, Rn, Rm



ADD Rd, Rn, imm8

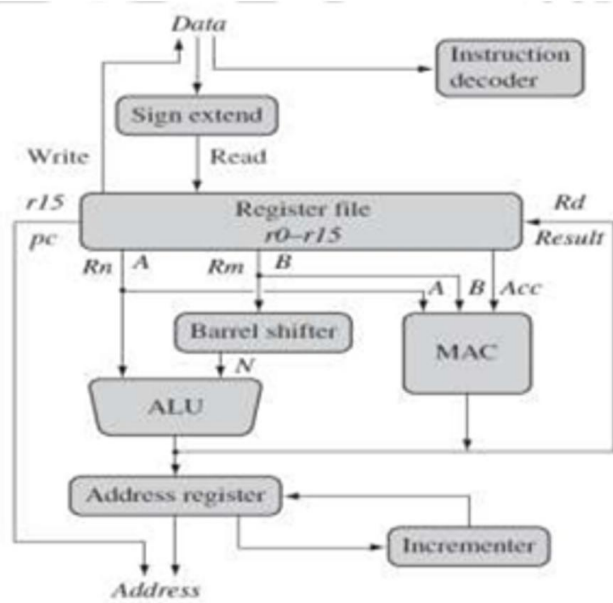


Figure 1.1 Data Flow Diagram of ARM